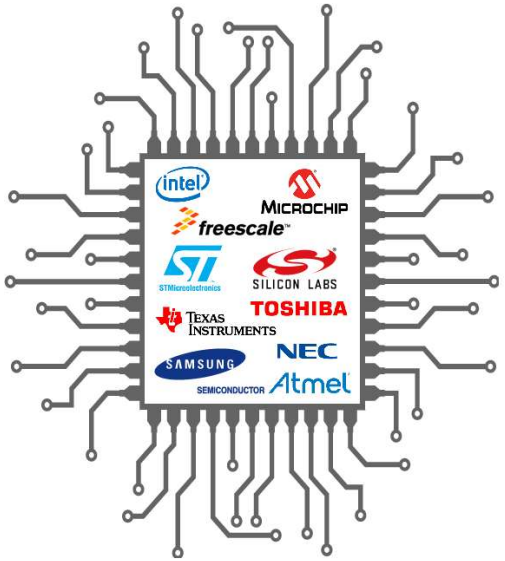
Pulse Width Modulation



Thareendhra Keerthi Wijayasiriwardhane

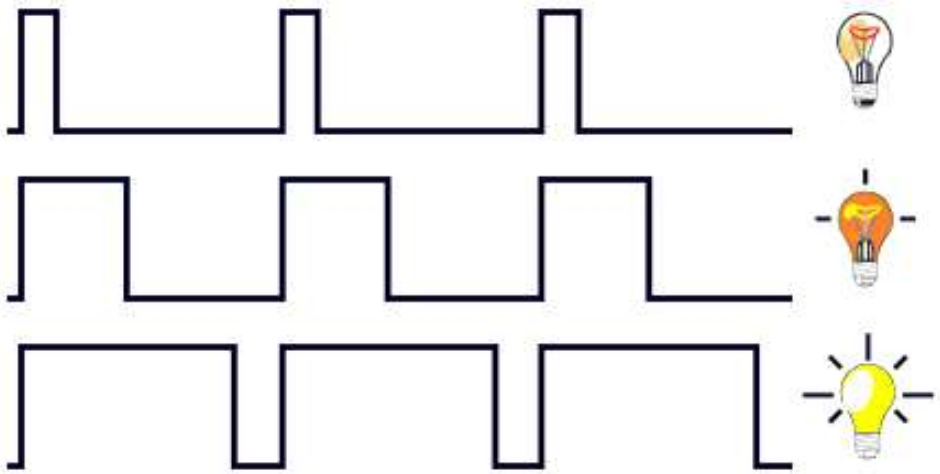
Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk

Learning Outcomes

- On completion of this session, you should be able to
  - Explain the need for PWM
  - Compute duty cycle and frequency of PWM outputs
  - Describe how Timers generate PWM outputs
  - Explain the functionality of Timer/Counter registers used for PWM
  - Construct microcontroller-based systems to produce analog like outputs by digital means

Pulse Width Modulation

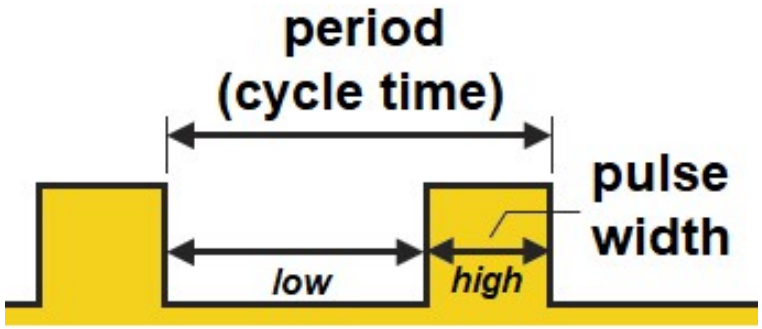
- A technique for getting an analog like output by digital means
- Toggles a digital output fast enough, making any attached device to react not to the individual pulses, but to the % of time that the output is high in one cycle



  - How fast?

Duty Cycle

- Percentage of time that output is high in one cycle



$$duty\ cycle = \frac{t_{high}}{t_{high} + t_{low}} \times 100\%$$

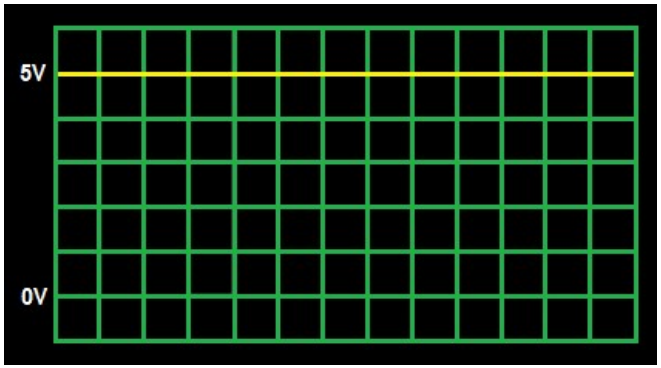
$$= \frac{t_{high}}{period} \times 100\%$$

$$= \frac{pulse\ width}{period} \times 100\%$$

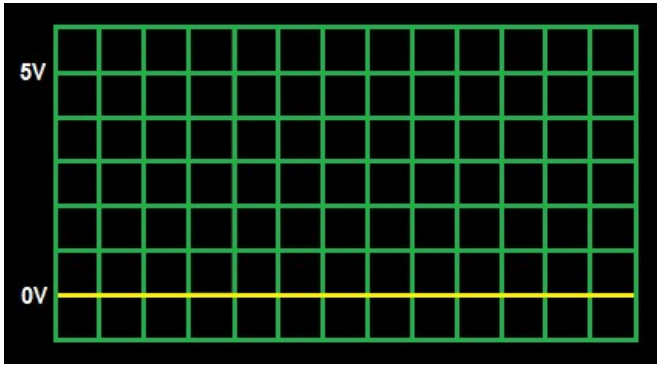
Average Voltage



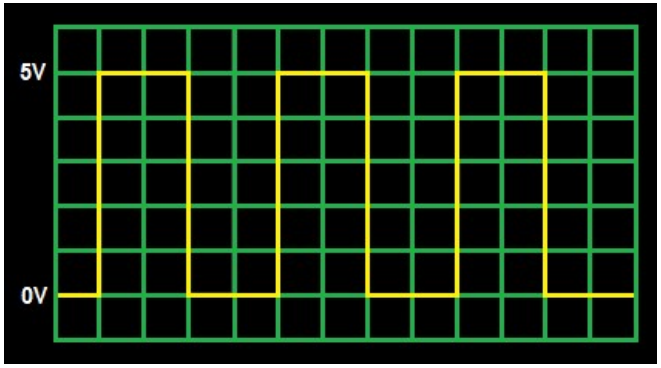








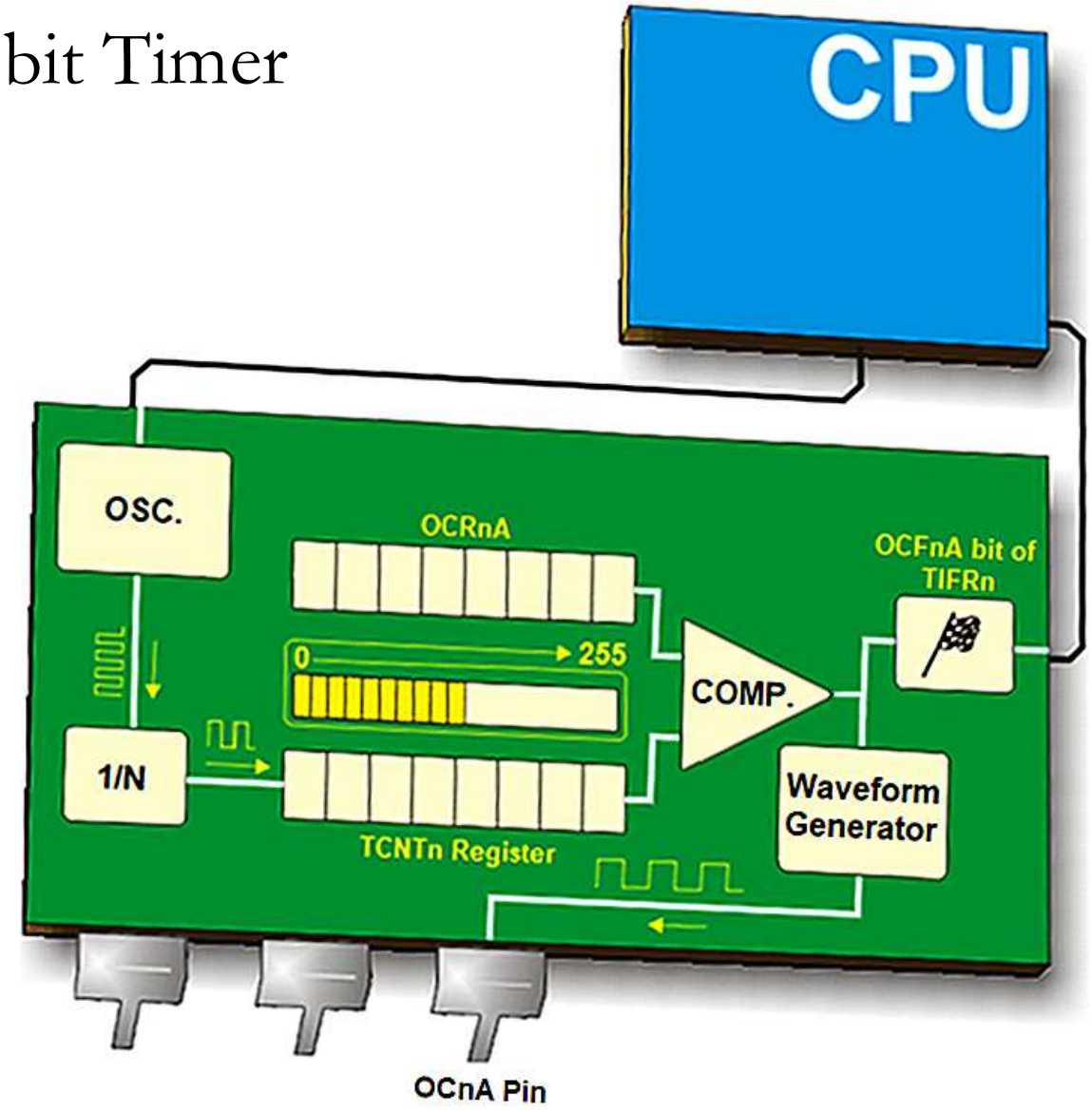




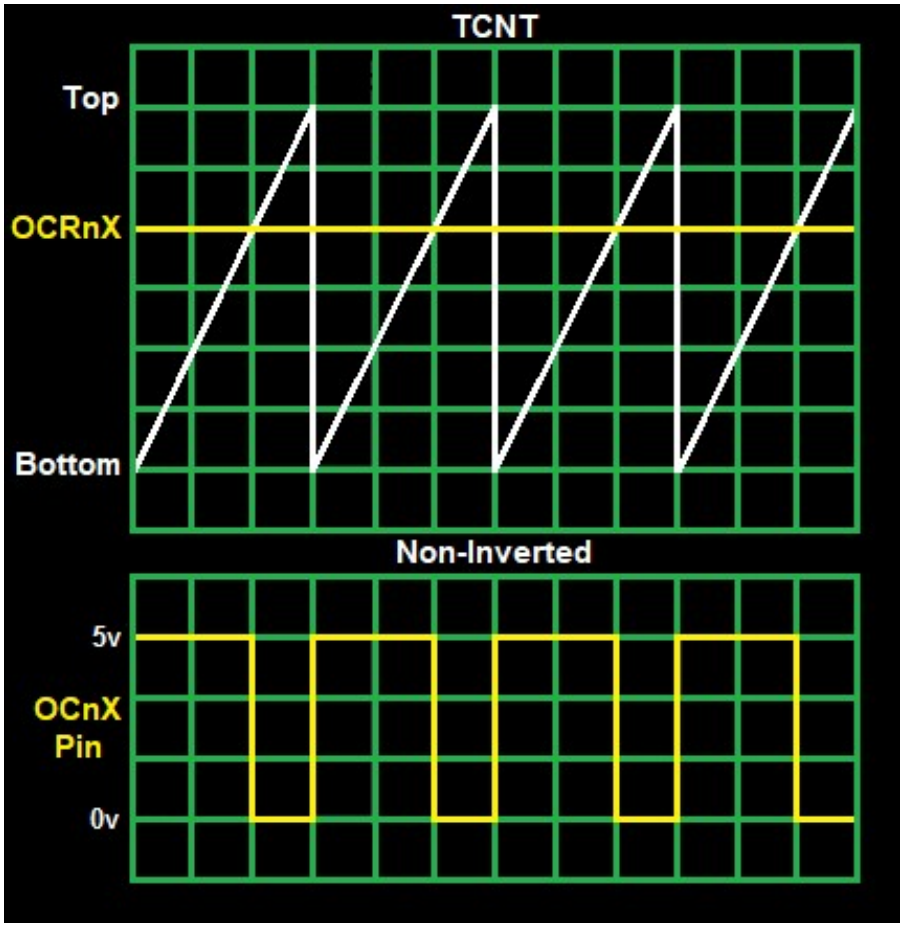


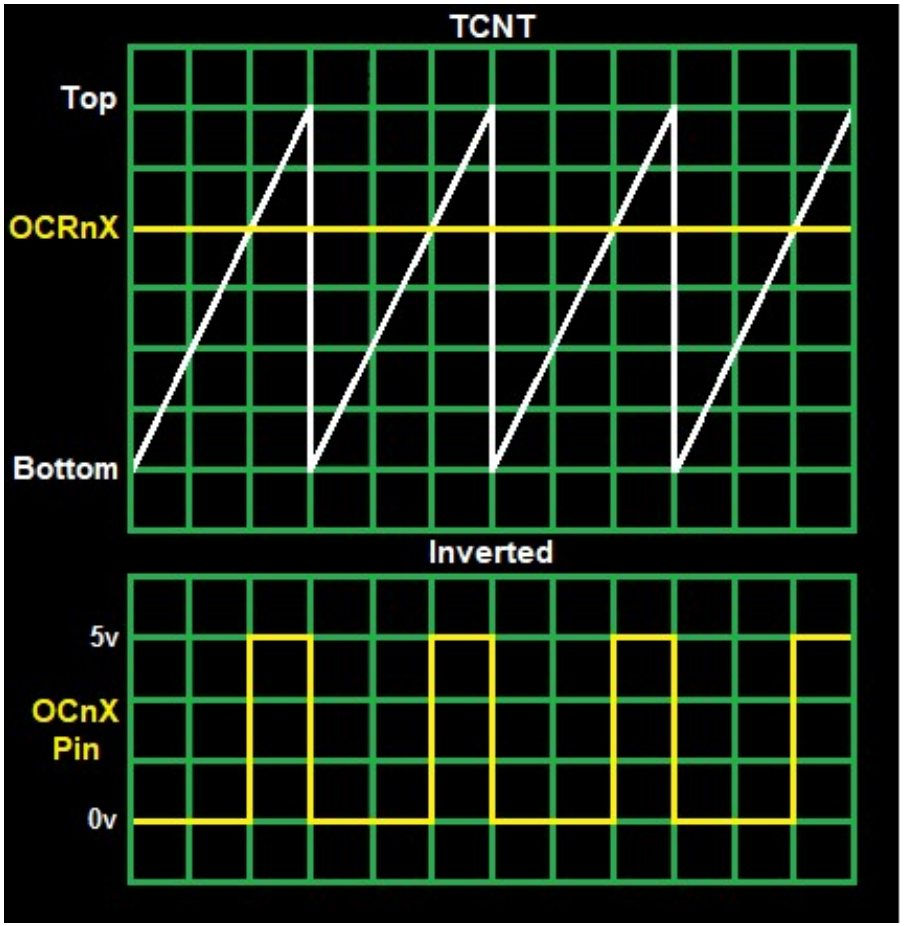
PWM Output Generation

- Using 8-bit Timer



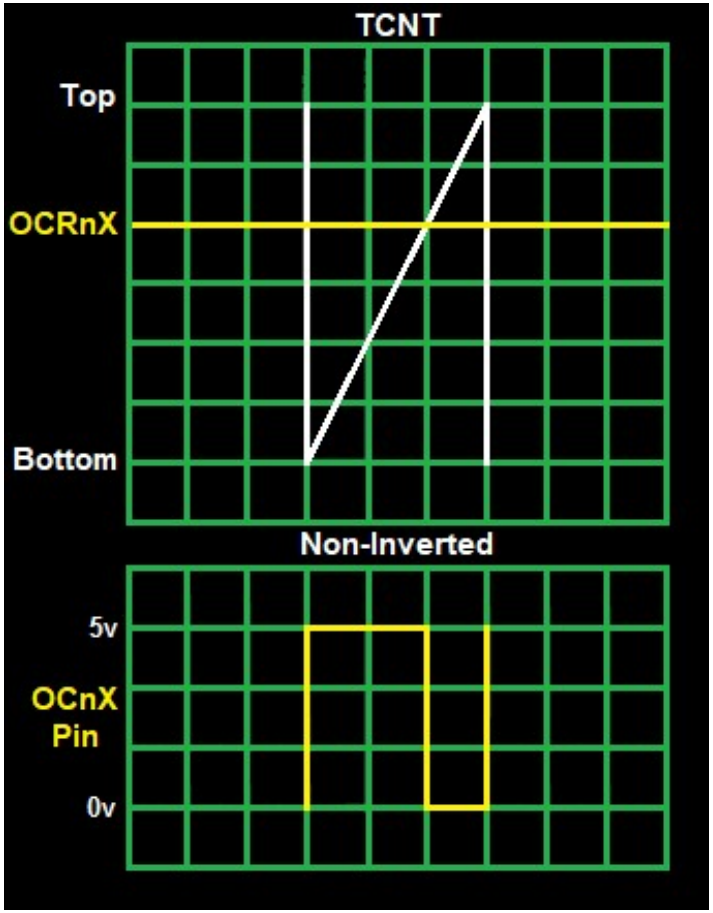
Waveform Generation





PWM Frequency

- In Non-Inverted mode, while Timer counts, if a compare match occurs, Waveform Generator clears OCnX pin and sets it when Timer comes to Bottom
- Waveform Generator generates a one PWM cycle



$$F_Timer = \frac{F_CPU}{prescale}$$

$$F_PWM = \frac{F_Timer}{(top + 1)} = \frac{F_CPU}{(top + 1) \times prescale}$$

PWM Frequency

- Determine prescale for required PWM frequency
  - For 8-bit Timers, top is $2^8 - 1$
  - For 16-bit Timers, top is $2^{16} - 1$

$$F_PWM = \frac{F_CPU}{(top + 1) \times prescale}$$

$$prescale = \frac{F_CPU}{(top + 1) \times F_PWM}$$

Timer / Counter Registers

- TCCRnB (Timer/Counter Control Register n B)

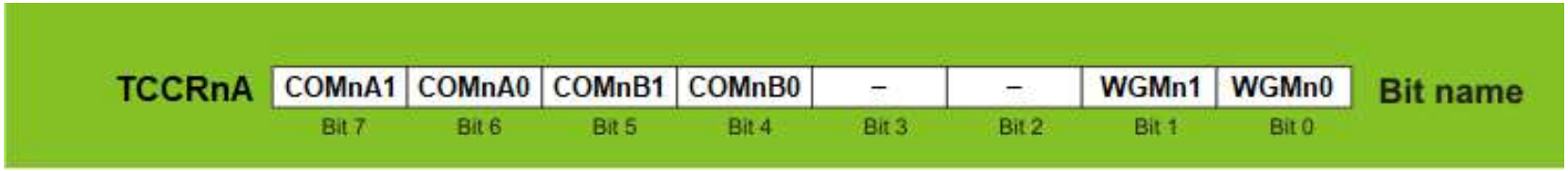


  - CSn2 (Clock Select), CSn1 and CSn0 bits select prescale

CSn2	CSn1	CSn0	Clock Source
0	0	0	Disconnect clock source
0	0	1	F_CPU
0	1	0	F_CPU/8
0	1	1	F_CPU/64
1	0	0	F_CPU/256
1	0	1	F_CPU/1024

Timer / Counter Registers

- TCCRnA (Timer/Counter Control Register n A)



  - WGMn1 (Waveform Generation Mode) and WGMn0 bits with WGMn2 bit of TCCRnB specify waveform generation mode

WGMn2	WGMn1	WGMn0	Mode
0	0	0	Normal
0	0	1	Phase Correct PWM
0	1	0	Clear Timer on Compare (CTC)
0	1	1	Fast PWM

Timer Modes

- Determines how timers behave
- Normal mode
  - Timer increments TCNTn (Timer/CouNTer) register from 0 to TOP, overflows to 0 and starts again
  - Timer sets TOVn (Timer OVerflow) flag bit of TIFRn (Timer Interrupt Flag Register) register when overflows
  - Timer interrupts when overflows if timer overflow interrupt is enabled
  - No PWM support

Timer / Counter Registers

- TIFRn (Timer Interrupt Flag Register) of Timer n



  - when TCNTn register overflows from its maximum value to 0, TOVn (Timer Overflow n) flag bit is set
  - Set TOVn flag bit to clear it (W1C)
- TIMSKn (Timer Interrupt MaSK Register) of Timer n



  - TOIEn (Timer Overflow Interrupt Enable n) bit enables (or disables) Timer overflow interrupt

Timer Modes

- CTC (Clear Timer on Compare) mode
  - TOP is programmable
  - Timer increments TCNTn (Timer/CouNTer) register from 0 to OCRnA (Output Compare Register), clears TCNTn to 0 and starts again
  - Timer sets OCFnX (Output Compare match Flag) bit of TIFRn (Timer Interrupt Flag Register) when compare match between TCNTn and OCRnX occurs
  - Timer interrupts when compare match occurs if output compare interrupt is enabled
  - No PWM support

Timer / Counter Registers

- OCRnA (Output Compare Register n A) and OCRnB
  - hold values to be compared against TCNTn register
  - If 8-bit, can hold a value between 0 and $2^8 - 1$
  - If 16-bit, can hold a value between 0 and $2^{16} - 1$
- TIFRn (Timer Interrupt Flag Register n)



  - When a compare match occurs between OCRnX and TCNTn registers, OCFnX (Output Compare match Flag n X) flag bit is set
  - Set OCFnX flag bit to clear it

Timer / Counter Registers

- TIMSKn (Timer Interrupt MaSK Register n)



  - OCIEnA (Output Compare match Interrupt Enable n A) and OCIEnB bits enable (or disables) Output Compare match A and B Interrupts, respectively

Timer Modes

- Fast PWM mode
  - Timer increments TCNTn (Timer/CouNTer) register from 0 to TOP, overflows to 0 and starts again
  - Timer clears OCnX (Output Compare) pin when compare match between OCRnX and TCNTn occurs during increment and sets it when TCNTn becomes 0
  - Duty cycle of PWM signal at OCnX pin

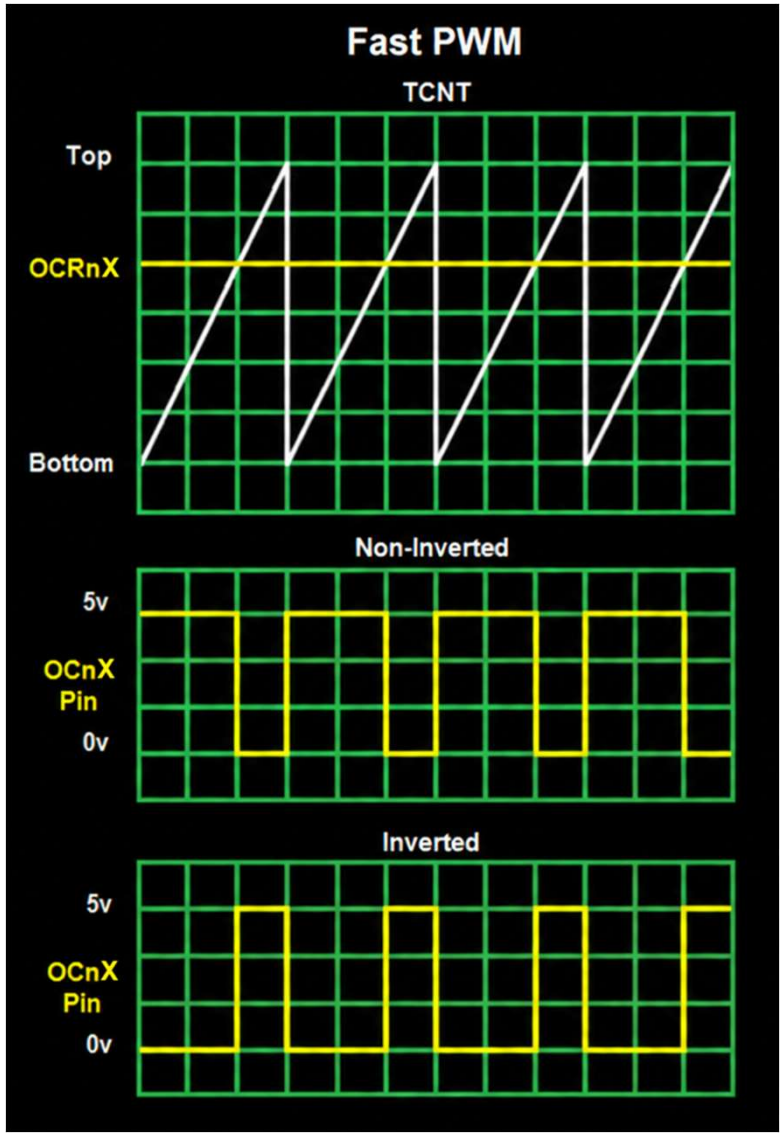
$$duty\ cycle = \frac{OCR + 1}{TOP + 1} \times 100\%$$

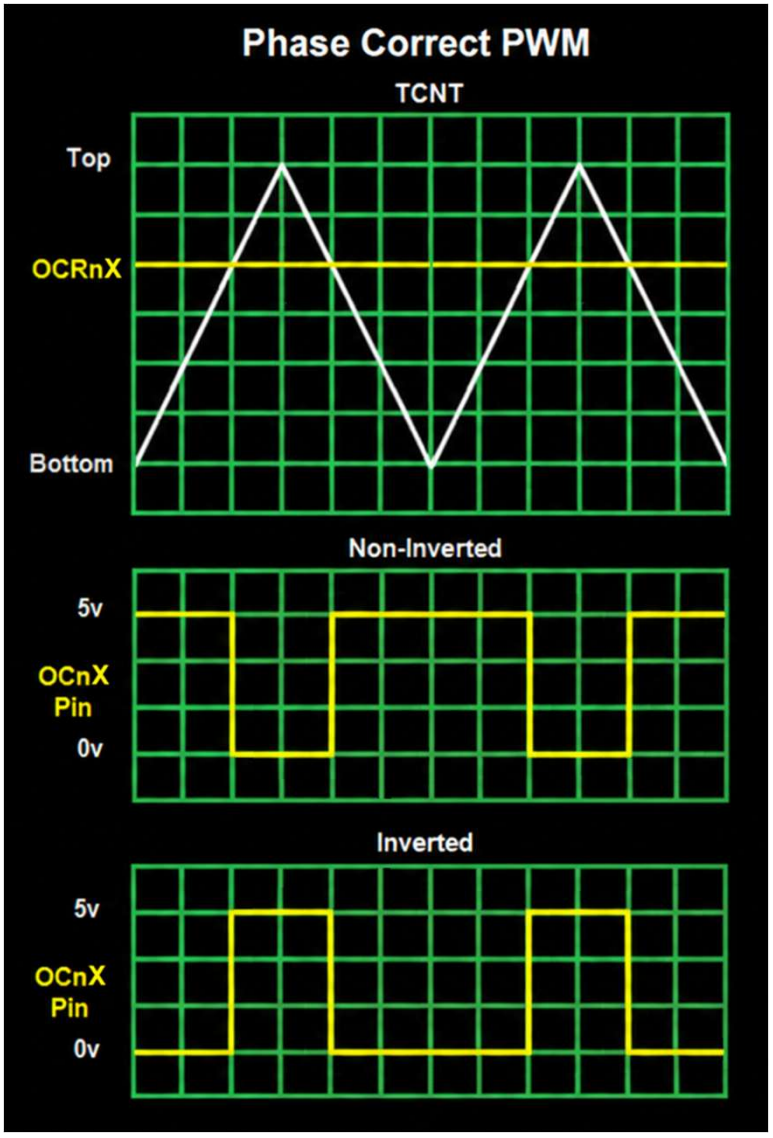
Timer Modes

- Phase Correct PWM mode
  - Timer increments TCNTn (Timer/CouNTer) register from 0 to TOP, decrements from TOP to 0 and repeats
  - Timer clears OCnX (Output Compare) pin when compare match between OCRnX and TCNTn occurs during increment and sets it when compare match occurs during decrement
  - Duty cycle of PWM signal at OCnX pin

$$duty\ cycle = \frac{OCR}{TOP} \times 100\%$$

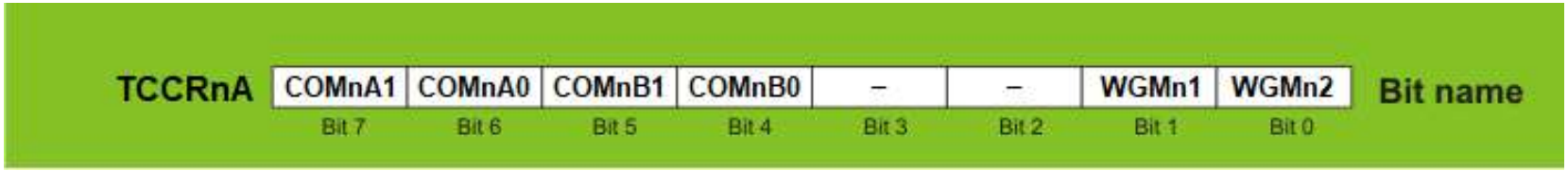
Fast PWM vs. Phase Correct PWM





Timer / Counter Registers

- TCCRnA (Timer/Counter Control Register n A)



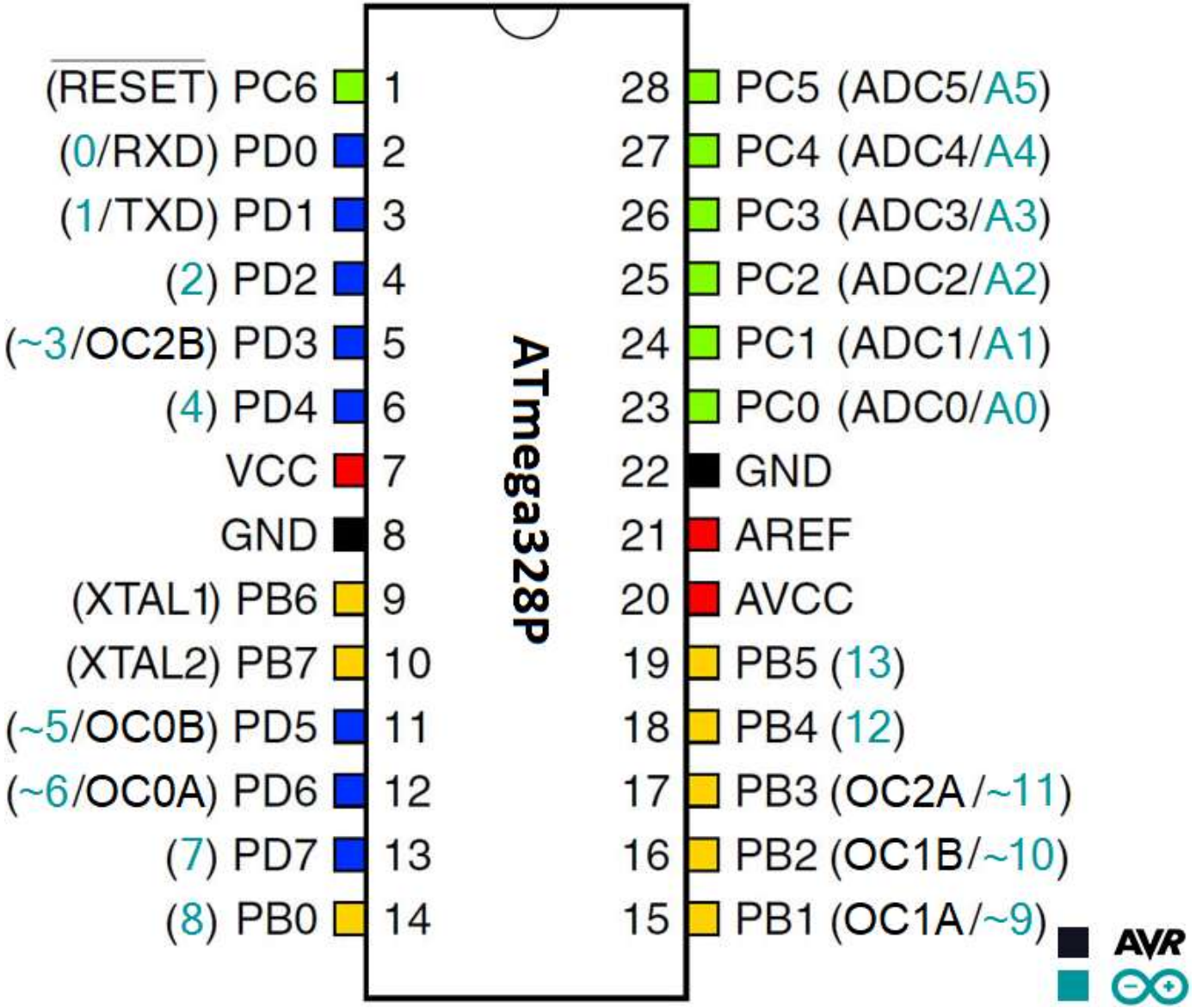
  - COMnA1 (Compare Output Mode), COMnA0, COMnB1 and COMnB0 bits specify compare output mode

COMnX1	COMnX0	Mode	Description
1	0	Non-Inverted	Clears OCnX on compare match & sets it at 0
1	1	Inverted	Sets OCnX on compare match & clears it at 0

System 15

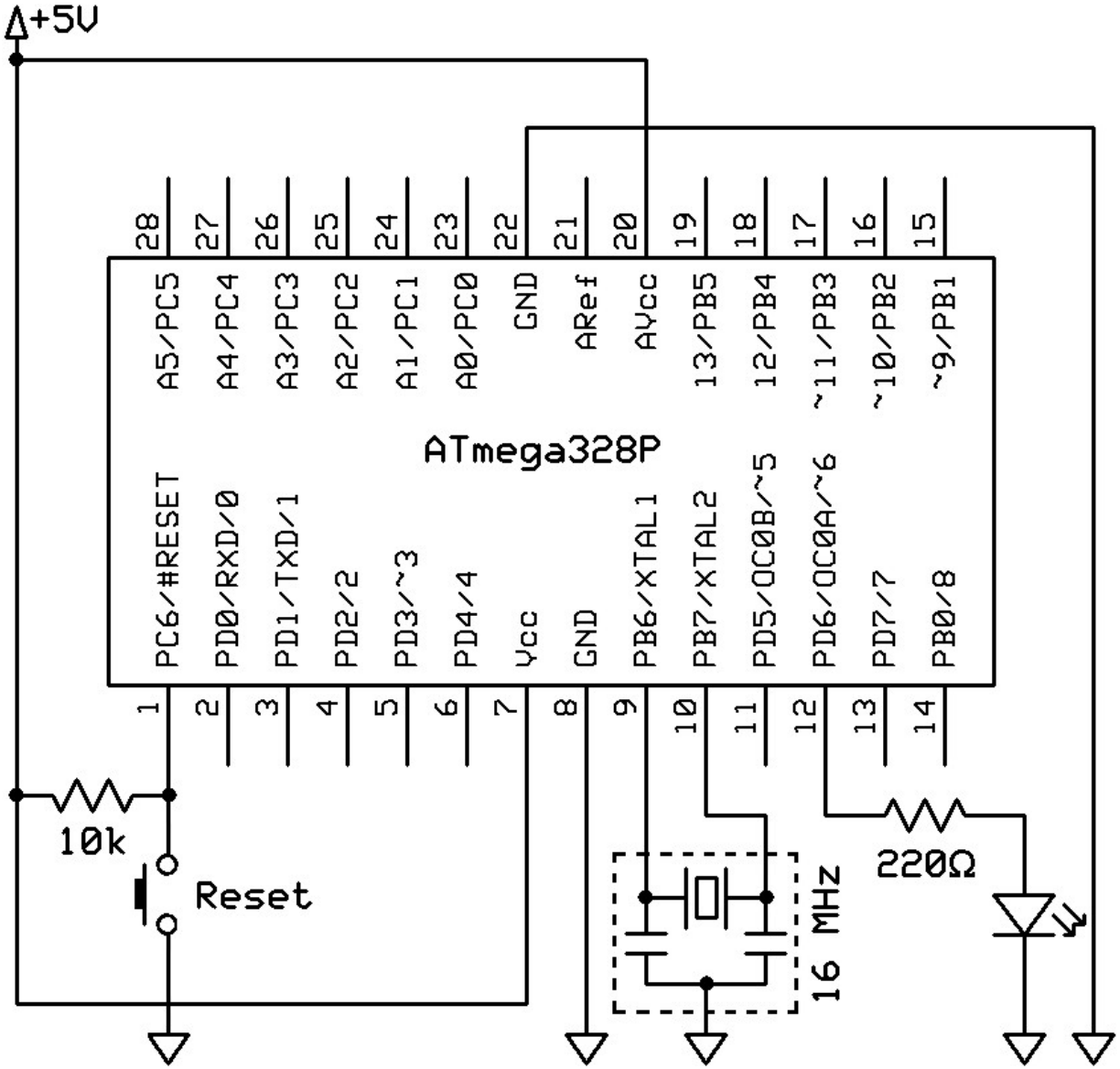
fader - Gradually fades the light of an LED again and again

ATmega328P PWM Pinout



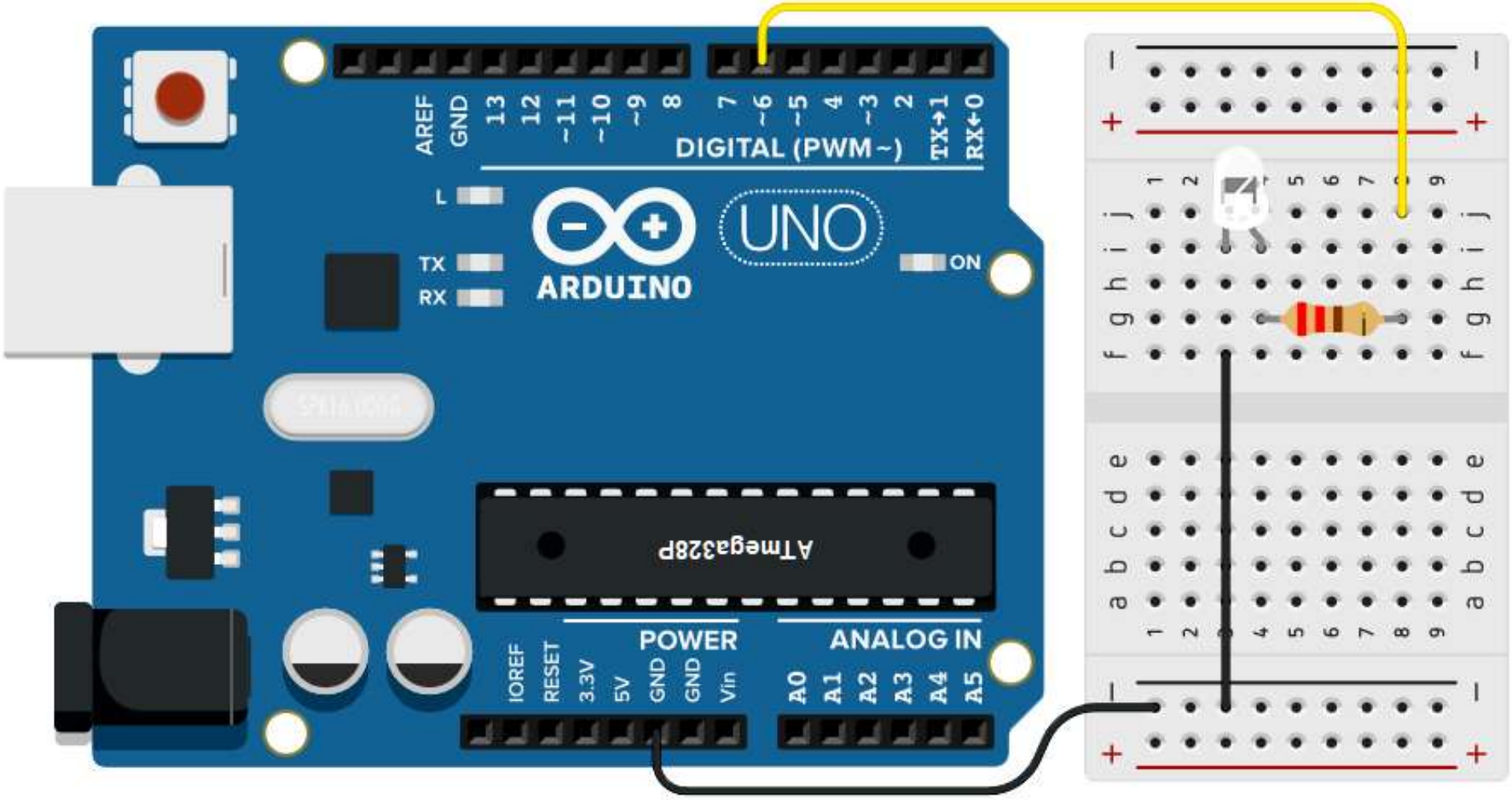
Hardware

- Schematic Diagram



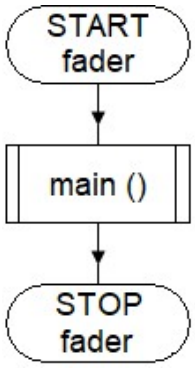
Hardware

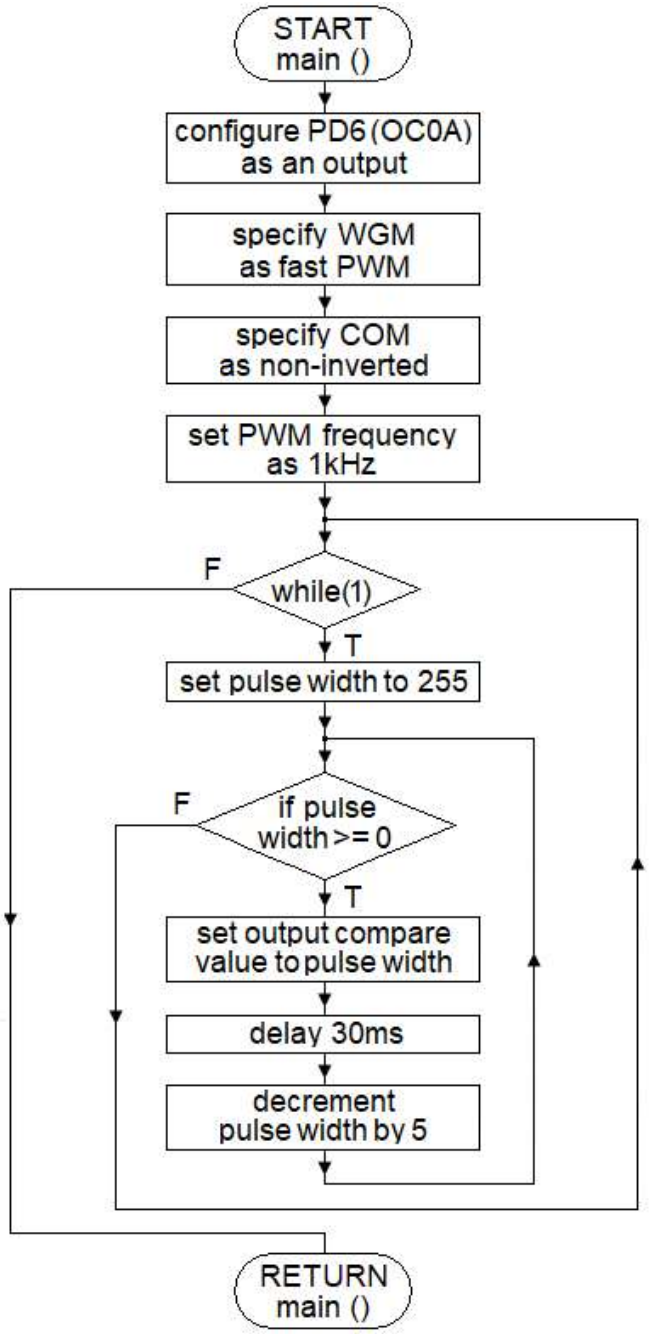
- Wiring Diagram



Firmware

- Algorithm



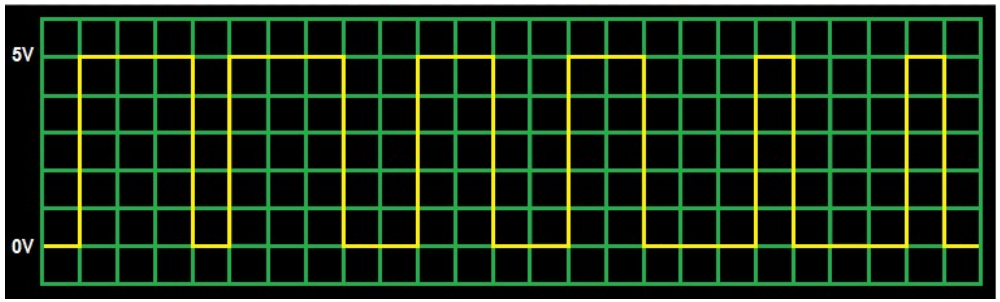


C Program

```c
// faderC.c

#include <avr/io.h>                                          // IO definitions
#define F_CPU 16000000UL                                     // define CPU clock speed for delay.h
#include <util/delay.h>                                      // definition of _delay_ms()

int main(void)
{
    DDRD = DDRD | 0b01000000;                                // configure PD6 (OC0A) pin as an output
    TCCR0B = TCCR0B & 0b11110111;                            //
    TCCR0A = TCCR0A | 0b00000011;                            // specify WGM as fast PWM
    TCCR0A = (TCCR0A & 0b10111111) | 0b10000000;             // specify COM as non-inverted
    TCCR0B = (TCCR0B & 0b11111011) | 0b00000011;             // set PWM frequency as 1kHz
    while (1)
    {
        for(int pulseWidth = 255; pulseWidth >= 0; pulseWidth -= 5)
        {
            OCR0A = pulseWidth;                              // set output compare value
            _delay_ms(30);                                   // delay 30ms
        }
    }
    return 0;
}
```



Using Hexadecimal Constants

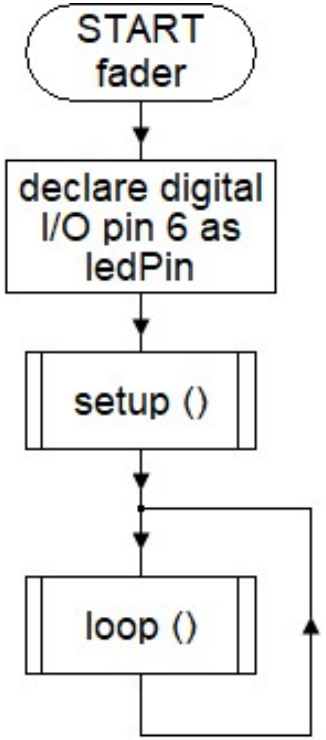
```
1   // faderC.c
2
3   #include <avr/io.h>                                     // IO definitions
4   #define F_CPU 16000000UL                                // define CPU clock speed for delay.h
5   #include <util/delay.h>                                 // definition of _delay_ms()
6
7   int main(void)
8   {
9       DDRD = DDRD | 0x40;                                 // configure PD6 (OC0A) pin as an output
10      TCCR0B = TCCR0B & 0xF7;                             //
11      TCCR0A = TCCR0A | 0x03;                             // specify WGM as fast PWM
12      TCCR0A = (TCCR0A & 0xBF) | 0x80;                    // specify COM as non-inverted
13      TCCR0B = (TCCR0B & 0xFB) | 0x03;                    // set PWM frequency as 1kHz
14      while (1)
15      {
16          for(int pulseWidth = 255; pulseWidth >= 0; pulseWidth -= 5)
17          {
18              OCR0A = pulseWidth;                         // set output compare value
19              _delay_ms(30);                              // delay 30ms
20          }
21      }
22      return 0;
23  }
```

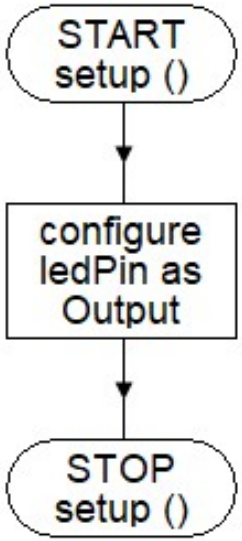
Using Labeled Constants

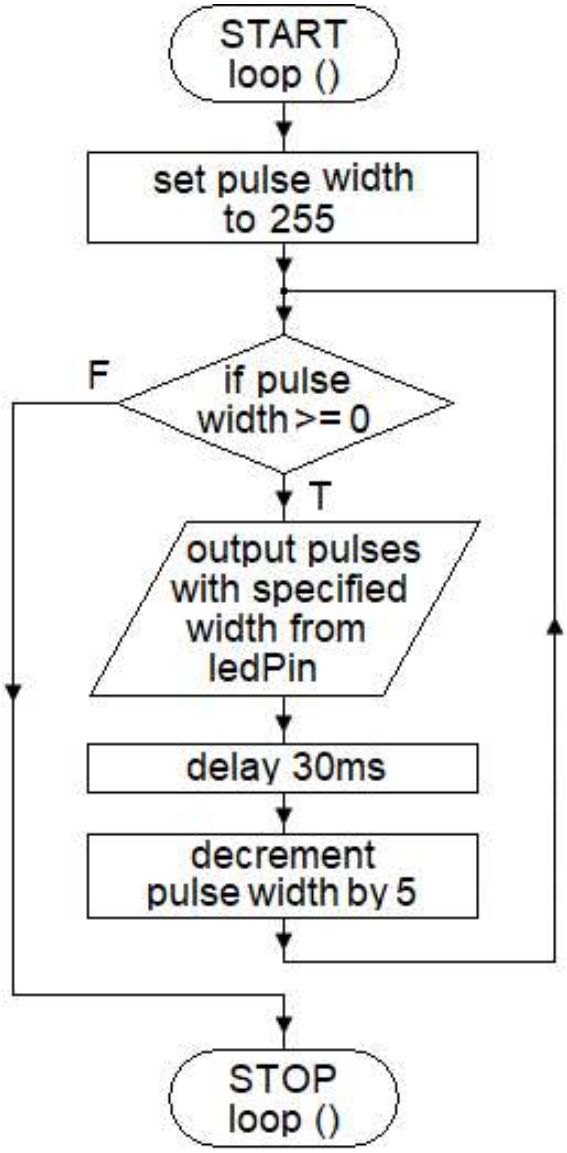
```
1   // faderC.c
2
3   #include <avr/io.h>                                    // IO definitions
4   #define F_CPU 16000000UL                               // define CPU clock speed for delay.h
5   #include <util/delay.h>                                // definition of _delay_ms()
6
7   int main(void)
8   {
9       DDRD = DDRD | (1<<DDD6);                           // configure PD6 (OC0A) pin as an output
10      TCCR0B = TCCR0B & ~(1<<WGM02);                     //
11      TCCR0A = TCCR0A | (1<<WGM01) | (1<<WGM00);         // specify WGM as fast PWM
12      TCCR0A = (TCCR0A & ~(1<<COM0A0)) | (1<<COM0A1);    // specify COM as non-inverted
13      TCCR0B = (TCCR0B & ~(1<<CS02)) | (1<<CS01) | (1<<CS00);   // set PWM frequency as 1kHz
14      while (1)
15      {
16          for(int pulseWidth = 255; pulseWidth >= 0; pulseWidth -= 5)
17          {
18              OCR0A = pulseWidth;                        // set output compare value
19              _delay_ms(30);                             // delay 30ms
20          }
21      }
22      return 0;
23  }
```

Firmware

- Algorithm for Arduino







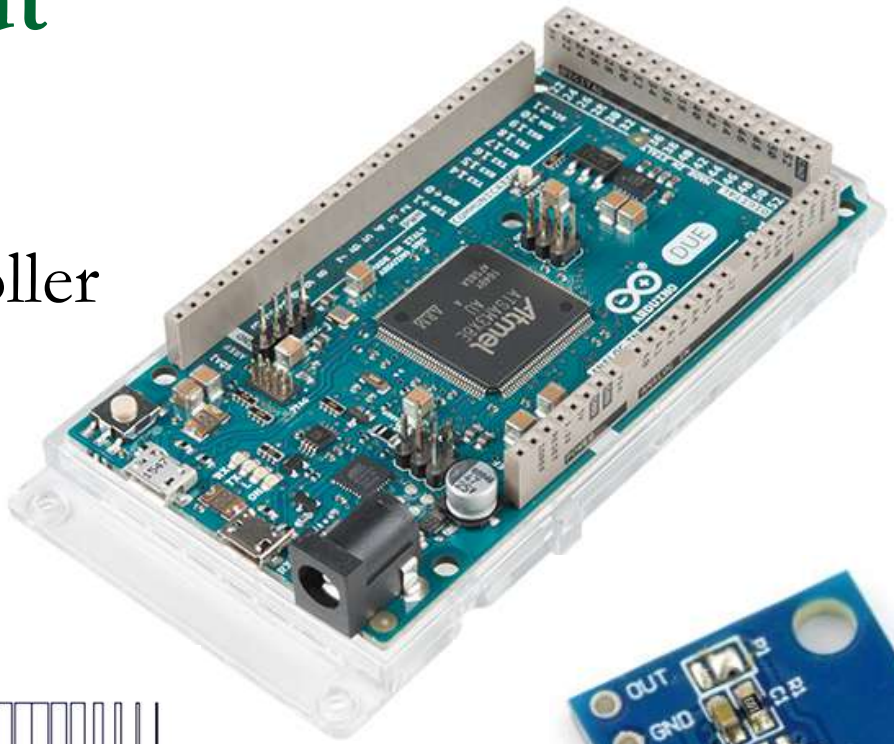
Firmware

- Source Code (using Arduino Wrapper Functions)

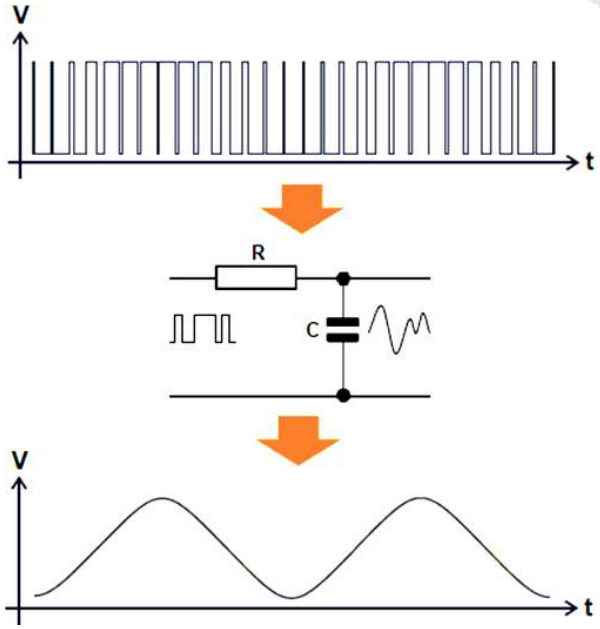
```
1 //fader.ino
2
3 int ledPin = 6;                        // declare digital I/O pin 6 as ledPin
4
5 void setup()
6 {
7   pinMode(ledPin, OUTPUT);            // configure LED connected pin as output
8 }
9
10 void loop()
11 {
12   for(int pulseWidth = 255; pulseWidth >=0; pulseWidth -= 5)
13   {
14     analogWrite(ledPin, pulseWidth); // output pulses with specified width
15     delay(30);                       // wait for 30ms
16   }
17 }
```

True Analog Output

- Three options
  - Switch to a microcontroller with built-in DAC
  - Add a DAC breakout
  - Use a RC filter







Questions